ALERT-03: Routing, Destinations, and Cost

> **Series:** ALERT — Alerting Strategy and Design | **Notebook:** 03 of 05 | **Created:** June 2026 | **Last Updated:** 10/15/2026

Overview

Once a problem fires, getting it to the right people without waste is a routing problem with a cost dimension. This notebook covers the **simple vs multi-step workflow** decision (and its billing implications), the destination landscape, and where the legacy alerting-profile path still applies. It orchestrates the WFLOW series rather than repeating it.

Table of Contents

1. Simple vs Multi-Step Workflows
2. The Routing Pattern
3. Destination Landscape
4. The Legacy Path
5. Cost Discipline

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS Gen3 with AutomationEngine (Workflows)
Prior reading	ALERT-01; routing depth in WFLOW-03/04

Requirement	Details
Upstream	Problems enriched with routing metadata (ALERT-01 §4)

1. Simple vs Multi-Step Workflows

This is the cost decision that the field most often gets wrong.

	Simple workflow	Multi-step workflow
Shape	**Exactly one task** — any trigger except Run Workflow, any action except Run JavaScript / Run Workflow / Approval Request	Trigger → conditions, multiple actions, enrichment, multiple teams
Billing	No workflow-hours — but each run can still bill Standard Function Calls and DQL query usage	Workflow-hours for every hour the workflow **exists**, from creation, whether it runs or not (about 720 a month), **plus** one Standard Function Call for every executed task
Use when	Filter problems and send to one channel	Routing to multiple teams, leaving comments, conditional logic, enrichment, centralised config

Default to simple workflows where possible. This may mean several simple workflows instead of one big one — that is the cheaper shape, because they do not consume workflow-hours. *Cheaper is not free:* the docs are explicit that "while simple workflows don't directly consume workflow hours, their execution can trigger the consumption of billable Dynatrace capabilities" — a Standard Function Call for a Slack message, or DQL query usage for a query inside the workflow. Reach for a multi-step workflow when you genuinely need conditional branching, multi-team routing, or a single centralised configuration; the capability is worth the cost when you need it, but do not pay it by default.

> **Two Dynatrace pages describe simple-workflow invocations differently.** The DPS billing page's worked example bills a simple workflow's notifications *"as Standard Function Calls"* (the capability formerly named AppEngine Functions). The upgrade guide (updated 09/30/2026) says each task execution *"counts as one AppEngine function invocation, and is billed as Automation Workflow"*. Both agree

> that a simple workflow consumes no workflow-hours. Check your rate card for which line the invocations land on.

A worked trap. A frequent pattern is *one routing workflow per team or area* that catches every problem for that area and posts to the team's chat channel. It looks like simple fan-out — but what decides the category is the **task count and action type**, not the apparent shape. One task with any action except Run JavaScript, Run Workflow, or Approval Request keeps the workflow simple — connector actions and HTTP Request included. A Run JavaScript task is excluded outright, and adding a second task tips the workflow into the billed multi-step category. That step does not undo itself: *"Once you start using the full functionality, there is no option to turn it off, even if you remove any additional tasks or actions."* The way back is a new workflow, or a rollback to an earlier version in the version history. Choose the action type before you build, not after.

> **Sources:**
>
> - Create a simple workflow (DT docs) — *"limited to only one task"*; *"Any action (except Run JavaScript, Run Workflow, Approval Request)"*; *"Once you start using the full functionality, there is no option to turn it off, even if you remove any additional tasks or actions."*
> - Automation consumption (DPS) (DT docs) — *"Workflow hours are the number of hours that a workflow has existed in your environment, measured since the point of its creation."*; *"24 hours × 14 days = 336 workflow hours"*; *"Each executed task"* consumes *"one Standard Function Call"*; *"billed as Standard Function Calls"*
> - Upgrade from Classic problem notification to simple workflows (DT docs) — *"each task execution counts as one AppEngine function invocation, and is billed as Automation Workflow"*
> - **Derived:** about 720 workflow-hours a month is the page's 24-hours-a-day rule over a 30-day month.

2. The Routing Pattern

1. Create a workflow with a **Problem trigger**, and set **Problem state** to *active or closed* so every open notification has a matching close.
2. Configure the trigger to **filter the problems** relevant to this channel, on the metadata you enriched upstream (team or area tags, service, severity).
3. Under *Advanced options*, enable **Wait for root cause analysis**. Without it the workflow can fire before the affected entities and tags your filter depends on have been

attached.

4. Add the **notification action** for the channel, and set up its **connection** if not already present.
5. Compose the message with Jinja over the problem record, for example `{{ event()['event.name'] }}`, and include `{{ problem_link() }}` so recipients reach the problem in one click.

Routing dimensions — severity, team/ownership, service, time of day — and escalation patterns are covered in depth in WFLOW-04. Two documented routes replace a side-table. Primary Grail tags (`primary_tags.*`) are copied from the alerting events onto the problem, so the trigger filters on them directly; a problem that groups several entities holds the deduplicated union of their values. And since SaaS 1.337, ownership is available on Smartscape nodes, so a workflow can look up the owning team with the Ownership `get_owners` action. That is a second task, which makes the workflow a standard one (§1).

> **Breaking — SaaS 1.348 (pre-release; staged tenant rollout planned from 09/22/2026): `event.severity` is no longer defaulted.** *"Davis events and problems no longer default `event.severity` to `3`."* If step 2 filters on severity, a filter that was silently matching the default stops matching once 1.348 reaches your tenant: *"Workflows with a Davis event/problem trigger that filter on `event.severity=3` expecting it to be defaulted, might need to be changed to filter for `Any` severity to keep being triggered."* Before routing on severity, check which of your event sources actually **set** one. Until 1.348 reaches your tenant, the defaulting behavior still applies and severity filtering works as described above.
>
> **Sources:** Upgrade from Classic problem notification to simple workflows (DT docs) — *"Enable Wait for root cause analysis. Without this option enabled, the workflow can trigger on a problem whose root cause and affected entities are still being assembled." "You can filter and template on them directly on the problem"*; *"The problem carries the deduplicated union."* SaaS 1.337 release notes (DT docs) — *"Ownership information is now available in Smartscape and for Smartscape nodes."* What's new in Dynatrace SaaS 1.348 (DT docs) — pre-release, read 09/24/2026.

3. Destination Landscape

Destination	Path	Notebook
Slack / Teams	Native workflow connector	WFLOW-03/04
PagerDuty / on-call	Native connector — use for fast-burn pages	WFLOW-04, SLO-04
Jira	Native connector — create/comment/assign issues	WFLOW-04
Jira Service Management	Native JSM connector (SaaS 1.343) — send Dynatrace events to JSM for alert management	WFLOW-04
ServiceNow	Native connector / HTTP Table API / ITOM app	ALERT-04
Email	Native action	WFLOW-03
xMatters	HTTP Request action to an xMatters endpoint (no connector); an existing classic integration keeps working (§4)	WFLOW-08
Anything else	HTTP action to a webhook	WFLOW-08

Match the destination to the urgency: fast-burn / acute → page (PagerDuty, on-call); slow-burn / steady → ticket (Jira, ServiceNow).

Migrating a classic notification: the official mapping

If you are converting classic problem notifications to workflows, this is the documented old→new correspondence:

Classic notification	Workflow equivalent
Ansible	Red Hat Ansible connector
Custom integration (generic webhook)	HTTP Request action
Email	Microsoft 365 / Email connector
Jira	Jira connector
PagerDuty	PagerDuty connector

Classic notification	Workflow equivalent
ServiceNow	ServiceNow connector
Slack	Slack connector
Microsoft Teams	Microsoft Teams connector

The three with no native connector

Trello, VictorOps, and xMatters have no native workflow connector. Opsgenie, which Atlassian is replacing with Jira Service Management, maps to the native JSM connector in the table above. Each becomes an HTTP-action rebuild against the destination's current API. A single HTTP Request task keeps the workflow *simple*; it becomes a billed standard workflow only if the payload needs a Run JavaScript step or a second task (§1). Budget for that case rather than discovering it.

Rebuild the payload against the destination's live API contract; do not port the classic webhook body verbatim.

> ⚠️ **A classic integration scoped by a Management Zone is only as durable as that Management Zone.** Workflows have no Management Zone filter. The upgrade guide describes the replacement: *"A workflow's Problem trigger filters problems directly with DQL matchers on the problem." "There is no separate filter object to create, name, and maintain, nor is there a one-management-zone-per-profile constraint."* If you are retiring Management Zones, rebuild those notifications as problem-triggered workflows first, filtered on entity tags or primary Grail tags. **MZ2POL-09** covers that conversion end to end, including the capability regressions.
>
> **Available (SaaS 1.343):** a dedicated **Jira Service Management connector** is part of the destination landscape — it sends Dynatrace events into JSM's alert management, distinct from the existing Jira (issue-tracking) connector. SaaS 1.343's rollout started **July 14, 2026**; treat it as the first-choice JSM path. The existing Jira connector and custom-webhook paths described in this section remain valid alternatives.
>
> **Sources:** Upgrade from Classic problem notification to simple workflows (DT docs) — the classic-to-workflow mapping table and the Management Zone replacement, quoted above; *"xMatters No dedicated connector. Use HTTP Request."* Create a simple workflow (DT docs) — *"Any action (except Run JavaScript, Run Workflow, Approval Request)"*. What's new in Dynatrace SaaS 1.343 (DT docs)

> *— "The Jira Service Management (JSM) Connector allows you to send Dynatrace information like problem events to Jira Service Management to create, update, or close alerts."*

4. The Legacy Path

Some integrations predate workflows and still rely on **classic problem notifications** driven by alerting profiles rather than the AutomationEngine. An existing **xMatters** integration is a common example. Workflows have no xMatters connector, and the upgrade guide maps xMatters to an HTTP Request action that posts the problem to an xMatters endpoint — the path a new xMatters integration should take.

If you have a classic problem-notification integration working, it keeps working: alerting profiles and problem notifications *"continue to work and are not being removed on a published schedule, but they'll not receive new capabilities."* New integrations should use workflows. The one caveat that does bite: if the integration is scoped by a **Management Zone**, it inherits that zone's lifetime (see §3). When you find a destination that "only works the classic way," check whether it exposes a webhook a workflow HTTP action can target before settling for the alerting-profile path.

> **Sources:** Upgrade from Classic problem notification to simple workflows (DT docs) — *"xMatters No dedicated connector. Use HTTP Request."*; *"They continue to work and are not being removed on a published schedule, but they'll not receive new capabilities."*

5. Cost Discipline

- **Prefer simple workflows.** Several cheap simple workflows beat one billed multi-step workflow when no conditional logic is needed.
- **One problem, one notification.** Let the Davis problem group related signals; do not also alert on the underlying raw metrics, or you double-notify and double-spend.
- **Filter early in the trigger.** A trigger that matches every problem and decides relevance later still evaluates on every problem.
- **Centralise only when it pays.** A single multi-step routing workflow is easier to govern but is billed; weigh that against many simple ones.

> **Sources:** Workflows (DT docs), Workflow actions — Jira / MS Teams (DT docs). Automation consumption (DPS) (DT docs) — *"Standard workflows consume workflow hours."*, Create a simple workflow (DT docs) — *"limited to only one task"*.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.